	Pimpri Chinchwad Education Trust's **Pimpri Chinchwad college of Engineering**

Assignment No. 8

Course:- Database Management System Laboratory(BIT24PC04)

Name:- Mahadev Suresh Jundale

PRN:- 125B2F007

PROBLEM STATEMENT:-

No SQL: Aggregation and Indexing: Design and Develop Queries using aggregation and indexing with suitable example using Mongo DB.

THEORY:-

CASE:- ECOMMERCE AND ORDER MANAGEMENT SYSTEM

AGGREGATION IN NOSQL

Introduction

Aggregation in MongoDB is used to process multiple documents and return computed results. It helps in performing operations such as filtering, grouping, calculating totals, averages, and sorting data. MongoDB uses an aggregation pipeline where documents pass through multiple stages to produce the final result.

Aggregation is commonly used in E-Commerce systems to calculate total sales, count orders, find average price, and group data by customers or products.

Types of Aggregation

1. $match (Filtering Documents)

The $match stage is used to filter documents based on conditions. It works similar to WHERE clause in SQL.

Syntax:- db.collection_name.aggregate([
{ $match: { condition } }
])

```
ecommerceDB> db.orders.aggregate([
| { $match: { quantity: { $gt: 1 } } }
| ])
[
  {
    _id: ObjectId('69d2af226821ac0c2e3682d8'),
    order_id: 5002,
    customer_id: 2,
    product_id: 102,
    quantity: 2,
    total: 50000
  },
  {
    _id: ObjectId('69d2af226821ac0c2e3682d9'),
    order_id: 5003,
    customer_id: 3,
    product_id: 103,
    quantity: 3,
    total: 6000
  }
]
ecommerceDB>
```

2. $group (Grouping Documents)

The $group stage groups documents by a specified field and performs calculations like sum, count, or average.

Syntax

```
db.collection_name.aggregate([
{
$group: {
_id: "$field",
result: { $sum: "$field" }
}
}
])
```

Example (Total quantity by product)

```
ecommerceDB> db.orders.aggregate([
| {
| $group: {
| _id: "$product_id",
| total_quantity: { $sum: "$quantity" }
| }
| }
| ])
[
  { _id: 102, total_quantity: 2 },
  { _id: 101, total_quantity: 1 },
  { _id: 103, total_quantity: 3 }
]
ecommerceDB>
```

3. $sum (Calculate Total)

$sum is used inside $group to calculate total values.

Syntax

```
{ $sum: "$field" }
```

Example (Total sales)

```
ecommerceDB> db.orders.aggregate([
| {
| $group: {
| _id: null,
| total_sales: { $sum: "$total" }
| }
| }
| ])
[ { _id: null, total_sales: 111000 } ]
ecommerceDB>
```

4. $avg (Average Value)

The $avg operator calculates average values.

Syntax

{ $avg: "$field" }

Example (Average product price)

```
ecommerceDB> db.products.aggregate([
| {
| $group: {
| _id: null,
| average_price: { $avg: "$price" }
| }
| }
| ])
[ { _id: null, average_price: 38500 } ]
ecommerceDB> |
```

5. $sort (Sorting Documents)

The $sort stage is used to sort documents in ascending or descending order.

Syntax

db.collection_name.aggregate([
{ $sort: { field: 1 } }
])

Example (Sort products by price descending)

```
ecommerceDB> db.products.aggregate([
| { $sort: { price: -1 } }
| ])
[
  {
    _id: ObjectId('69d2af156821ac0c2e3682d4'),
    product_id: 101,
    name: 'Laptop',
    price: 52000,
    stock: 10
  },
  {
    _id: ObjectId('69d2af156821ac0c2e3682d5'),
    product_id: 102,
    name: 'Mobile',
    price: 25000,
    stock: 20
  }
]
ecommerceDB>
```

6. $project (Select Fields)

The $project stage is used to display selected fields.

Syntax

db.collection_name.aggregate([
{ $project: { field1: 1, field2: 1 } }
])

Example (Show name and price only)

```
ecommerceDB> db.products.aggregate([
| { $project: { name: 1, price: 1 } }
| ])
[
  {
    _id: ObjectId('69d2af156821ac0c2e3682d4'),
    name: 'Laptop',
    price: 52000
  },
  {
    _id: ObjectId('69d2af156821ac0c2e3682d5'),
    name: 'Mobile',
    price: 25000
  }
]
```

7. $limit (Limit Results)

Limits the number of documents returned.

Syntax

db.collection_name.aggregate([
{ $limit: number }
])

Example

```
ecommerceDB> db.products.aggregate([
| { $limit: 2 }
| ])
[
  {
    _id: ObjectId('69d2af156821ac0c2e3682d4'),
    product_id: 101,
    name: 'Laptop',
    price: 52000,
    stock: 10
  },
  {
    _id: ObjectId('69d2af156821ac0c2e3682d5'),
    product_id: 102,
    name: 'Mobile',
    price: 25000,
    stock: 20
  }
]
ecommerceDB>
```

8. $lookup (Aggregation Join)

Used to combine data from multiple collections.

Syntax

db.collection.aggregate([
{
$lookup: {
from: "collection",
localField: "field",
foreignField: "field",
as: "output"
}
}
])

Example

```
ecommerceDB> db.orders.aggregate([
| {
| $lookup: {
| from: "customers",
| localField: "customer_id",
| foreignField: "customer_id",
| as: "customer_details"
| }
| }
| ])
[
  {
    _id: ObjectId('69d2af226821ac0c2e3682d7'),
    order_id: 5001,
    customer_id: 1,
    product_id: 101,
    quantity: 1,
    total: 55000,
    customer_details: [
      {
        _id: ObjectId('69d2aefd6821ac0c2e3682d1'),
        customer_id: 1,
        name: 'Rahul Sharma',
        email: 'rahul@gmail.com',
        city: 'Mumbai'
      }
    ]
  },
  {
    _id: ObjectId('69d2af226821ac0c2e3682d8'),
    order_id: 5002,
    customer_id: 2,
    product_id: 102,
    quantity: 2,
    total: 50000,
    customer_details: [
      {
        _id: ObjectId('69d2aefd6821ac0c2e3682d2'),
        customer_id: 2,
        name: 'Priya Patil',
        email: 'priya@gmail.com',
        city: 'Mumbai'
      }
    ]
  },
  {
    _id: ObjectId('69d2af226821ac0c2e3682d9'),
    order_id: 5003,
    customer_id: 3,
    product_id: 103,
    quantity: 3,
    total: 6000,
    customer_details: []
  }
]
```

Advantages of Aggregation

- Performs complex calculations
- Groups data efficiently
- Improves data analysis
- Reduces multiple queries
- Works on large datasets
- Flexible pipeline processing

Disadvantages of Aggregation

- Complex syntax for beginners
- Slower on very large datasets without index
- High memory usage
- Difficult to debug long pipelines
- Requires aggregation knowledge

Use of Aggregation in E-Commerce

- Calculate total sales
- Find top selling products
- Count orders per customer
- Average product price
- Group orders by product
- Generate sales reports

INDEXING IN NOSQL

Introduction

Indexing in MongoDB is used to improve the performance of queries. An index stores a small portion of data in an easy-to-search format, allowing MongoDB to find documents quickly without scanning the entire collection. Indexing is useful when working with large datasets such as customers, products, and orders in an E-Commerce system.

Types of Indexing in MongoDB

1. Single Field Index

Single field index is created on one field. It improves search performance for queries using that field.

Syntax

db.collection_name.createIndex({field: 1})

Example

```
ecommerceDB> db.customers.createIndex({customer_id: 1})
customer_id_1
```

2. Compound Index

Compound index is created on multiple fields. It is used when queries filter using more than one field.

Syntax

db.collection_name.createIndex({field1: 1, field2: -1})

Example

```
ecommerceDB> db.products.createIndex({category: 1, price: -1})
category_1_price_-1
ecommerceDB>
```

3. Unique Index

Unique index ensures that duplicate values are not allowed in a field.

Syntax

db.collection_name.createIndex({field: 1}, {unique: true})

Example

```
ecommerceDB> db.customers.createIndex({email: 1}, {unique: true})
email_1
ecommerceDB>
```

4. Text Index

Text index is used for searching string content like product names or descriptions.

Syntax

db.collection_name.createIndex({field: "text"})

Example

```
ecommerceDB> db.products.createIndex({name: "text"})
name_text
ecommerceDB>
```

5. Multikey Index

Multikey index is used when indexing array fields.

Syntax

db.collection_name.createIndex({array_field: 1})

Example

```
ecommerceDB> db.products.createIndex({tags: 1})
tags_1
ecommerceDB>
```

Show Indexes

Displays all indexes created in a collection.

db.products.getIndexes()

```
ecommerceDB> db.products.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { category: 1, price: -1 }, name: 'category_1_price_-1' },
  {
    v: 2,
    key: { _fts: 'text', _ftsx: 1 },
    name: 'name_text',
    weights: { name: 1 },
    default_language: 'english',
    language_override: 'language',
    textIndexVersion: 3
  },
  { v: 2, key: { tags: 1 }, name: 'tags_1' }
]
ecommerceDB>
```

Drop Index

Removes an index.

```
db.products.dropIndex({price: 1})
```

Advantages of Indexing

- Improves query performance
- Faster data retrieval
- Reduces database scan time
- Efficient sorting
- Better filtering of records
- Useful for large datasets

Disadvantages of Indexing

- Uses extra storage space
- Slows insert operations
- Slows update operations
- Index maintenance required
- Too many indexes reduce performance

Use of Indexing in E-Commerce System

- Search customer by email
- Find product by category
- Filter orders by date
- Sort products by price
- Fast product search
- Improve order lookup speed

CONCLUSION

Aggregation and indexing in MongoDB are important features used to analyze and retrieve data efficiently. Aggregation helps in processing large amounts of data by performing operations like filtering, grouping, sorting, and calculating totals. Indexing improves query performance by allowing faster data access without scanning the entire collection.

In an E-Commerce system, aggregation is useful for generating sales reports, finding top-selling products, and analyzing customer orders, while indexing helps in quick searching of customers, products, and orders. Together, aggregation and indexing improve performance, reduce query time, and make the database more efficient for handling large datasets.